DeepSeek推理性能跃迁4.5倍，ACL Graph 技术解密

作者

时间



CANN

前言

模型	Eager模式	ACL Graph	加速比
Qwen3-30B	150.33ms	20.2ms	7.4x
DeepSeek-V3.1	185.6ms	41.2ms	4.5x

注：本性能数据采集基于Atlas A2产品

为什么使用ACL Graph能提升这么多性能?



ACL Graph场景分析

维度	说明
适用场景	Host-bound 场景
问题场景	推理场景中，算子通常较小、执行时间短
核心矛盾	CPU 调度速度跟不上 NPU 执行速度
后果	NPU 在等待下一次执行前形成调度空泡（Bubble），端到端耗时增大
解决方案	推出 ACL Graph 解决方案：一次捕获，多次执行
典型模型	DeepSeek-V3：61 层、每层 20+ 算子，气泡累积成显著延迟
典型实践（DeepSeek/Qwen3-MoE）	Prefill：保持 eager；Decode：根据 exe_mode 切换到 ACL Graph模式
Prefill 不适合 ACL Graph 的原因	shape 变化大，单次计算量大且时间长，host 下发不是瓶颈

CANN

目录

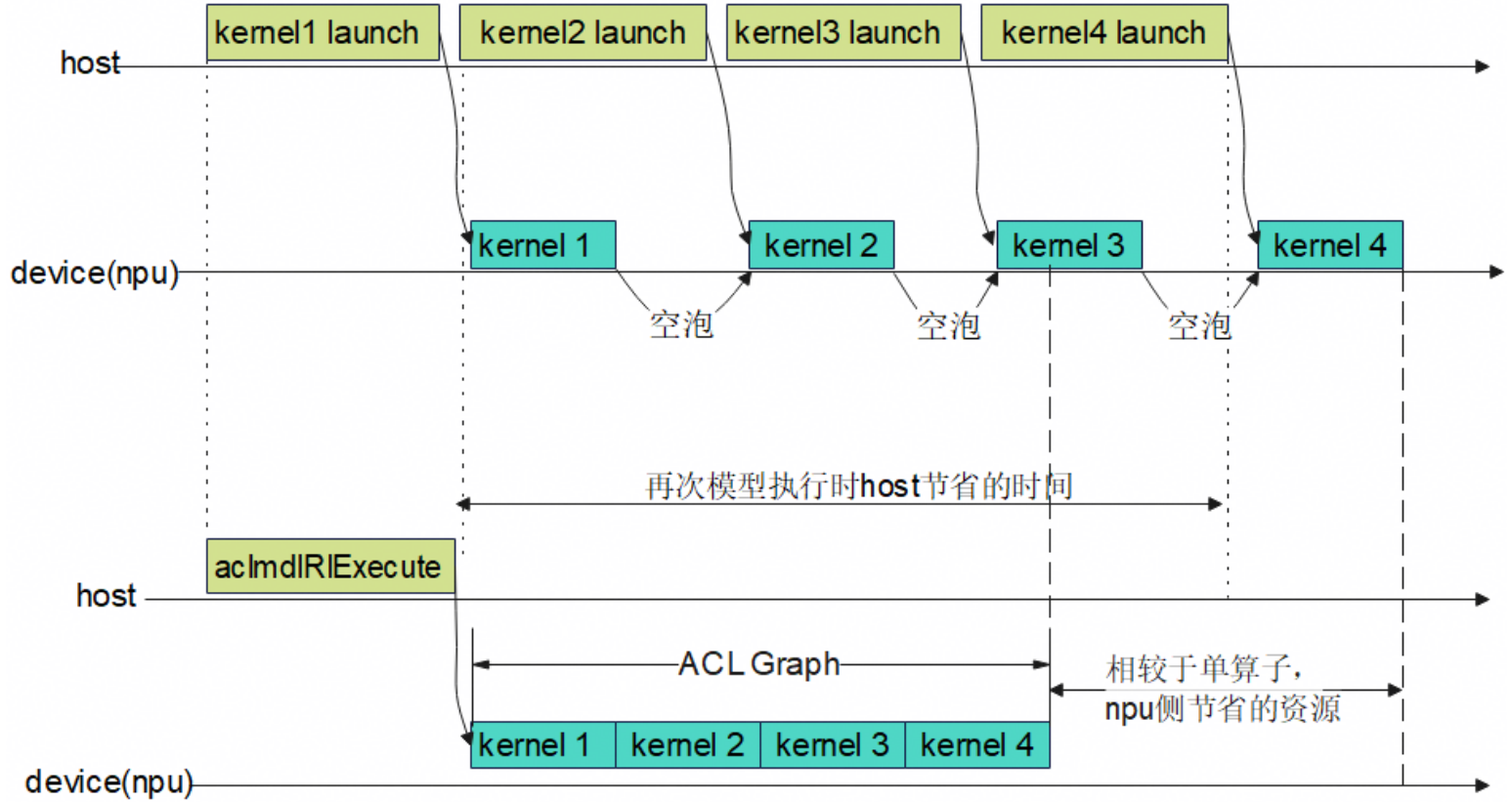
Part 1 整体方案介绍

Part 2 关键设计介绍

Part 3 未来演进方向

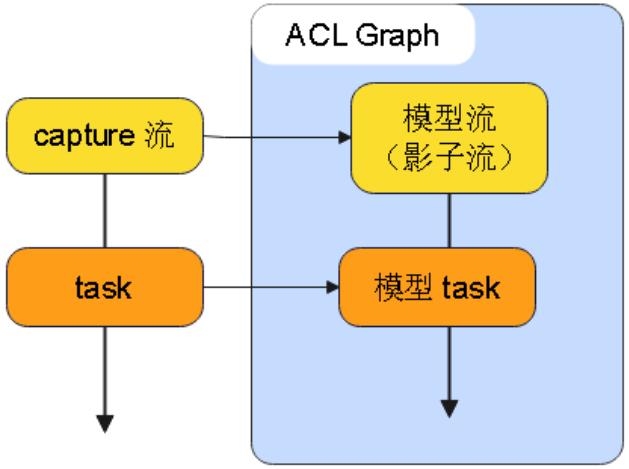
ACL Graph背景及原理

- 当前eager模式下由于任务是**即下发即执行**，当发生**Host-Bound**时**npu会产生很多空泡**，直接影响npu的利用率。
- 为解决这一问题，提出ACL Graph方案，分为**capture + replay两个阶段**，**第一阶段捕获**用户下发的任务，组装成一个ACL Graph，**第二阶段为replay阶段**，不用重复下发模型任务，host只需要反复replay即可达到任务重新执行的目的。



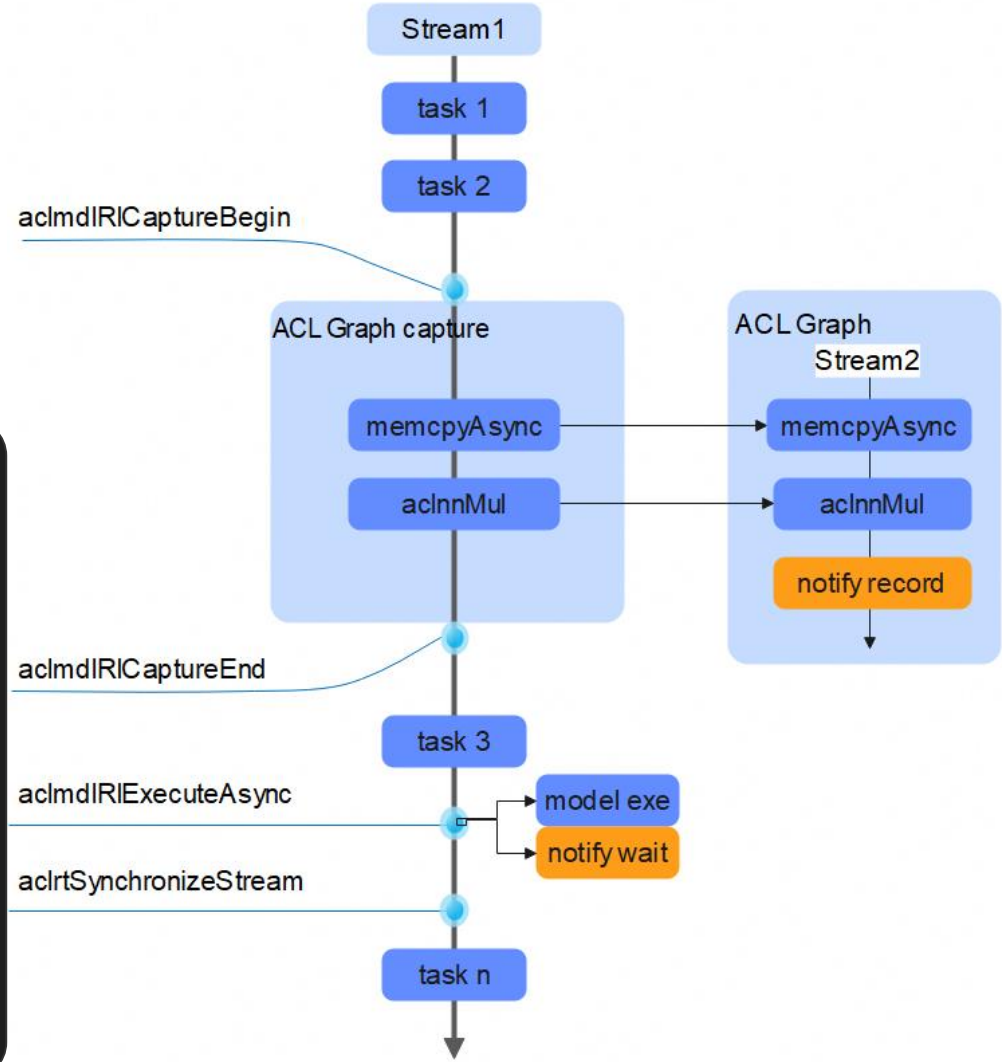
单流捕获原理

在单算子流task下发过程中插入aclmdlRICaptureBegin与aclmdlRICaptureEnd，
把begin、end之间下发的task捕获到同一ACL Graph中。




```
// capture 阶段
// 捕获开始
CHECK_ERROR(aclmdlRICaptureBegin(stream,
    ACL_MODEL_RI_CAPTURE_MODE_GLOBAL));
CHECK_ERROR(aclrtMemcpyAsync(dstAddr, size, srcAddr, size,
    ACL_MEMCPY_HOST_TO_DEVICE, stream));
aclnnMul(mulWorkspaceAddr, mulWorkspaceSize, mulExecutor, stream);
// 捕获结束
CHECK_ERROR(aclmdlRICaptureEnd(stream, &modelRI));

// replay 阶段
CHECK_ERROR(aclmdlRIExecuteAsync(modelRI, stream));
CHECK_ERROR(aclrtSynchronizeStream(stream));
```



CANN

ACL Graph的两个阶段

capture阶段：
- Runtime保存capture stream上下发的任务参数
- 申请device内存用于保存用户的kernel args
- 构造对应的任务描述符
- End capture时校验用户构造的ACL Graph的完整性与正确性
- 不改变任务的执行序与依赖关系，以用户下发为准
- capture阶段不真正执行任务

replay阶段：
- 为模型流选定硬件队列，并指向任务描述符
- 每次replay时trigger硬件队列执行指向的任务

总结：ACL Graph把多次 kernel 启动和依赖关系预先“录制”成一张图，之后一次性提交执行，从而大幅降低 CPU 启动开销、提高 NPU 利用率和执行效率。

目录

Part 1 整体方案介绍

Part 2 关键设计介绍

Part 3 未来演进方向

多流捕获

多流捕获依赖两条流之间的event record、wait。
在捕获状态的stream1上下发event record，stream2上下发event wait
这样就能把stream2也捕获到同一个ACL Graph中，之后直到end capture。

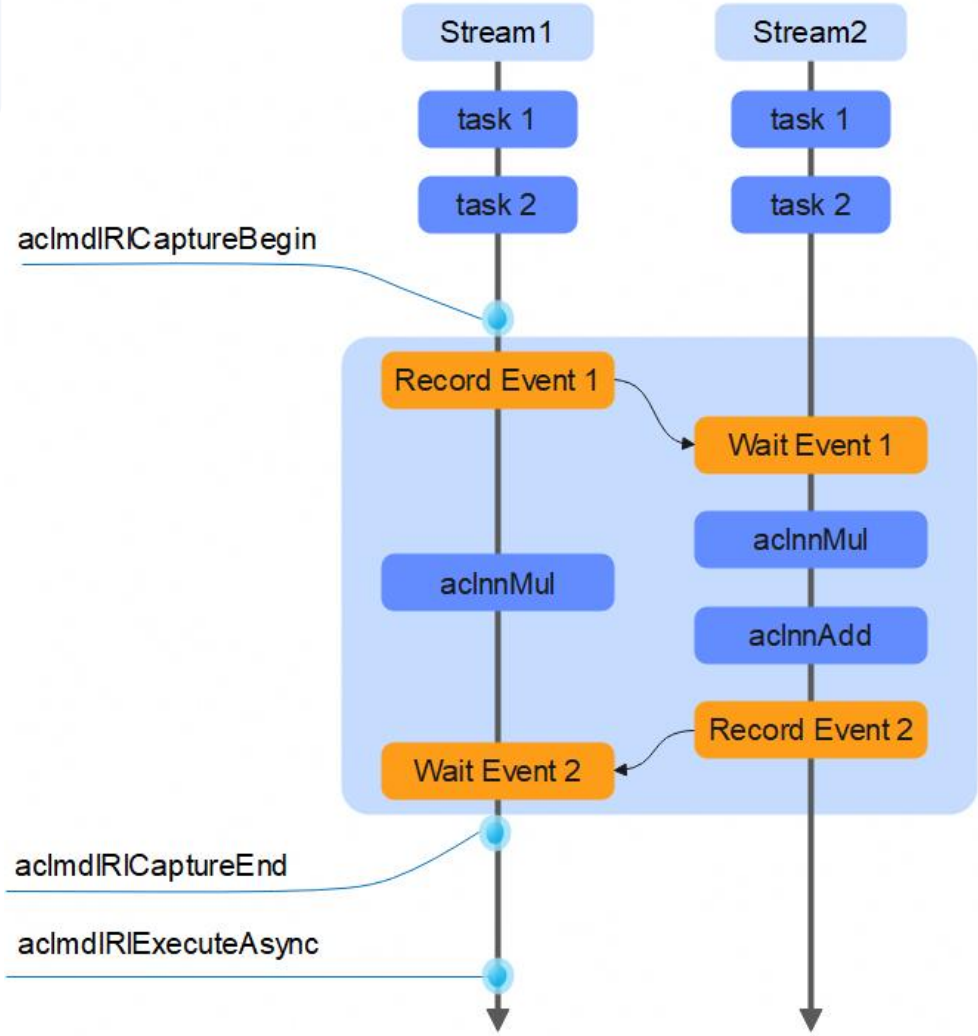
```
// capture阶段
// 捕获开始
CHECK_ERROR(aclmdlRICaptureBegin(stream1,
    ACL_MODEL_RI_CAPTURE_MODE_GLOBAL));
CHECK_ERROR(aclrtRecordEvent(event1, stream1));
CHECK_ERROR(aclrtStreamWaitEvent(stream2, event1));

aclnnMul(mulWorkspaceAddr, mulWorkspaceSize, mulExecutor, stream1);
aclnnMul(mulWorkspaceAddr2, mulWorkspaceSize2, mulExecutor2, stream2);
aclnnAdd(mulWorkspaceAddr2, mulWorkspaceSize2, mulExecutor2, stream2);

CHECK_ERROR(aclrtRecordEvent(event2, stream2));
CHECK_ERROR(aclrtStreamWaitEvent(stream1, event2));

// 捕获结束
CHECK_ERROR(aclmdlRICaptureEnd(stream, &modelRI));

// replay阶段
CHECK_ERROR(aclmdlRIExecuteAsync(modelRI, stream));
CHECK_ERROR(aclrtSynchronizeStream(stream));
```



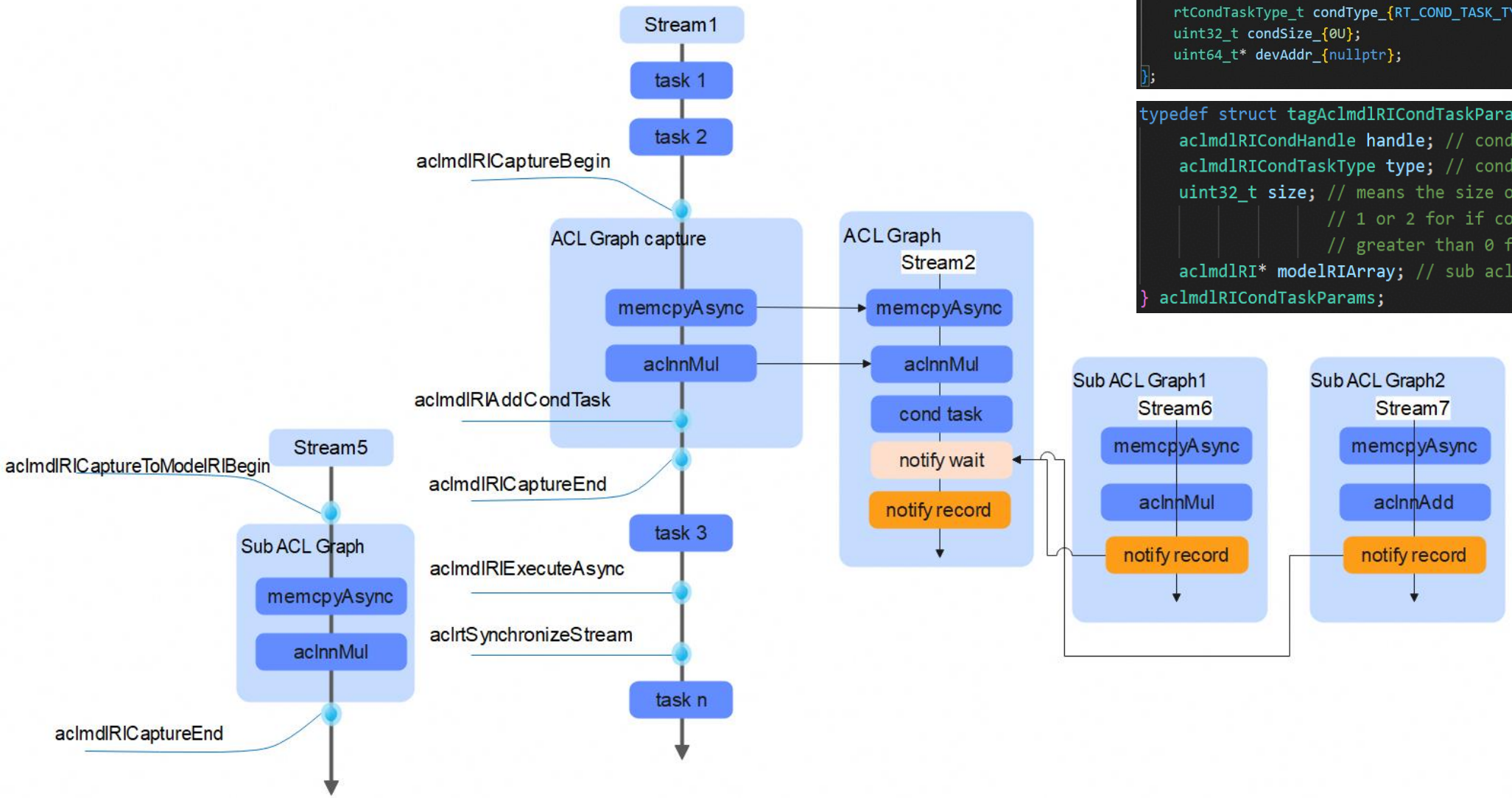
CANN

条件子图的背景及动机

- 传统ACL Graph的痛点

问题	说明
静态执行	图一旦创建结束，执行路径完全固定，无法根据运行时计算结果改变
被迫返回到 CPU	遇到条件分支或循环决策时，必须中断 NPU 执行，返回 CPU 判断后重新提交新的图执行
性能损失	频繁的 NPU <->CPU 同步会占用 CPU 资源，增加启动开销

条件子图捕获

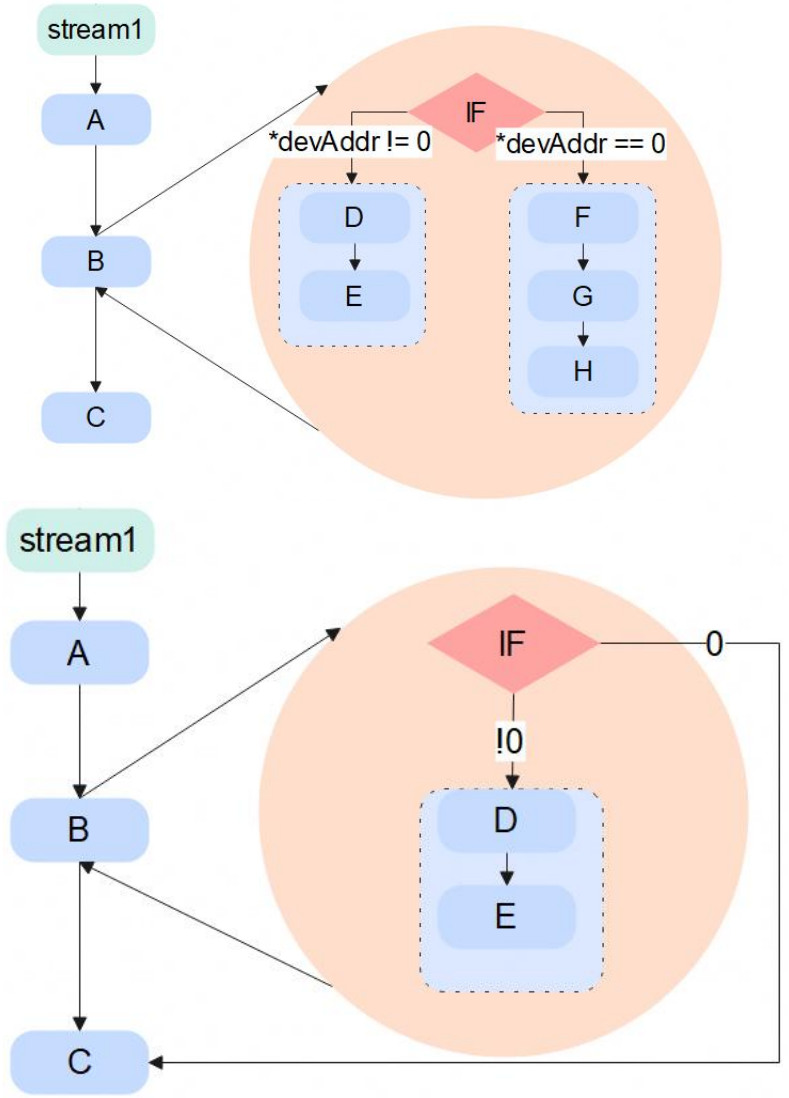



```
class CondHandle : public NoCopy {
private:
    uint32_t defaultValue_{0U}; // 父模型执行时的默认条件值
    // RT_COND_HANDLE_ASSIGN_DEFAULT表示模型执行时，条件值初始化为defaultValue_
    rtCondHandleFlag_t flag_;
    rtCondTaskType_t condType_{RT_COND_TASK_TYPE_MAX}; // condition type
    uint32_t condSize_{0U};                            // 子模型个数
    uint64_t* devAddr_{nullptr};                       // device ptr，用于存储条件值
};
```

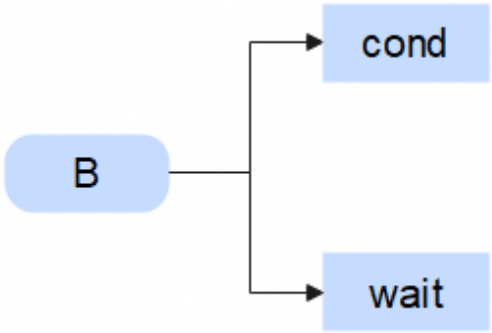
```
typedef struct tagAclmdlRICondTaskParams {
    aclmdlRICondHandle handle; // condition handle
    aclmdlRICondTaskType type; // condition type
    uint32_t size; // means the size of modelRIArray,
                   // 1 or 2 for if condition, 1 for while condition,
                   // greater than 0 for switch condition
    aclmdlRI* modelRIArray; // sub acl graph, output parameter.
} aclmdlRICondTaskParams;
```


CANN

IF条件子图



1、每次replay，每个条件节点执行0个或者1个子图
2、实际上任务B是由cond task + notify wait组成

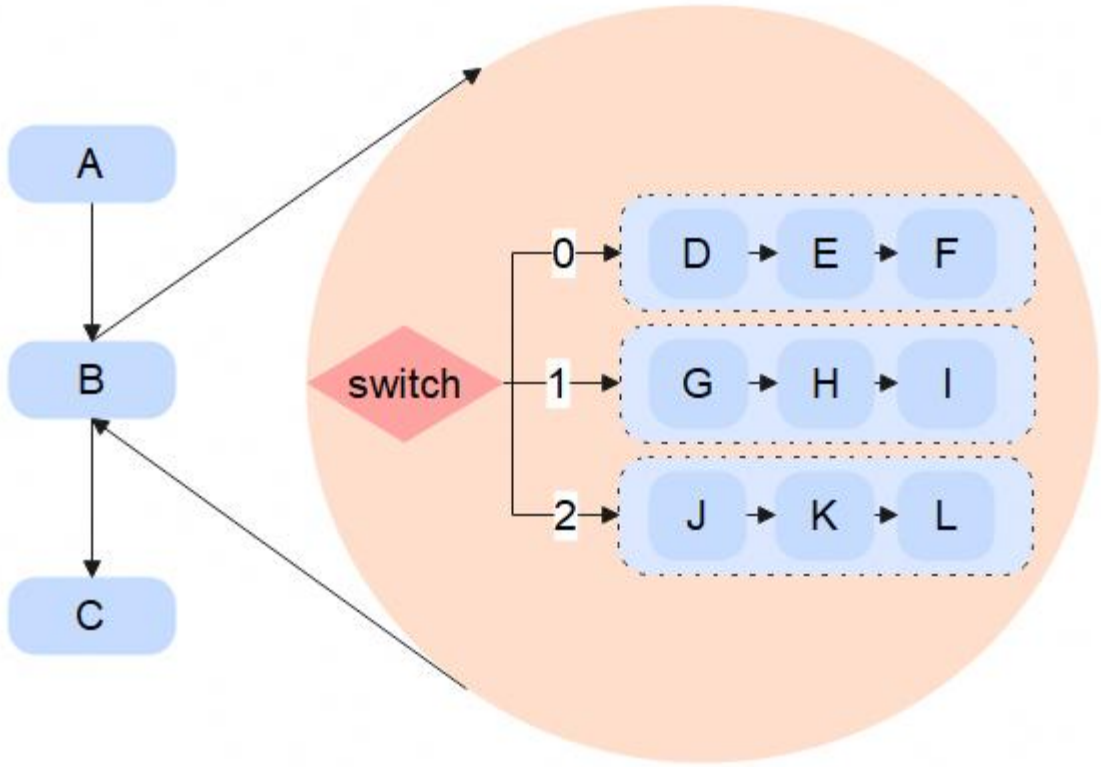


根据条件值拉起待执行的子图

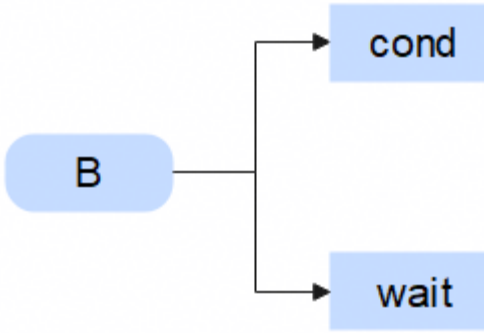
等待条件子图执行完成

CANN

SWITCH条件子图



1、每次replay，每个条件节点执行0个或者1个子图
2、实际上任务B是由cond task + notify wait组成

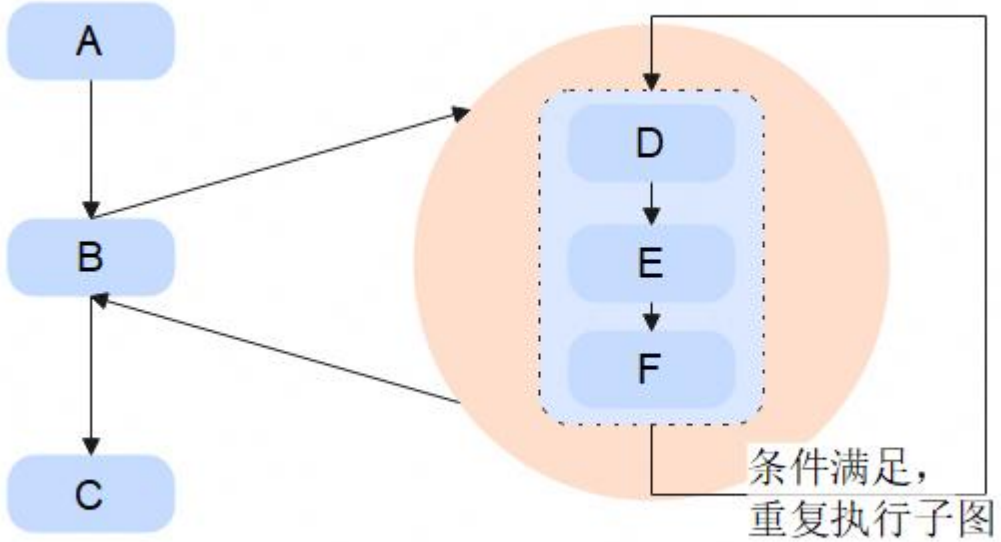


根据条件值拉起待执行的子图

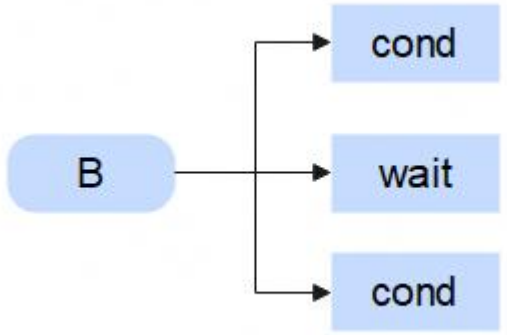
等待条件子图执行完成

CANN

WHILE条件子图



1、每次replay，每个条件节点的子图执行零次或多次
2、实际上任务B是由cond task + notify wait组成



根据条件值拉起待执行的子图

等待条件子图执行完成

根据条件值判断
是否再次执行子图

CANN

条件节点

某些业务场景下，图中的任务无法在 graph capture阶段确定是否执行，只能在graph运行时根据前序任务计算结果才能确定。
传统ACL Graph需要CPU介入，选择执行另一个ACL Graph再执行，增加了一次调度开销。
条件节点其实是捕获多个条件子模型，在执行到条件节点时，根据当前的条件值，选择不同的子模型。子模型执行结束后，条件节点所在模型流才可以正常继续执行。

```
CHECK_ERROR(aclmdlRICaptureBegin(stream, ACL_MODEL_RI_CAPTURE_MODE_THREAD_LOCAL));
// 获取当前父model对象
CHECK_ERROR(aclmdlRICaptureGetInfo(stream, &status, &parentModelRI));

// 基于父model创建条件句柄
CHECK_ERROR(aclmdlRICondHandleCreate(parentModelRI, 0, ACL_MODEL_RI_COND_HANDLE_ASSIGN_DEFAULT, &condHandle));
// 获取条件值地址，用户直接修改地址的值，添加条件任务节点
CHECK_ERROR(aclmdlRICondHandleGetCondPtr(condHandle, &condDevPtr));
setHandle<<<1, 1, 0, stream>>>(condDevPtr);

CHECK_ERROR(aclmdlRIAddCondTask(params, stream, 0));
// 捕获子model
CHECK_ERROR(aclmdlRICaptureToModelRIBegin(subStream, subModels[0], ACL_MODEL_RI_CAPTURE_MODE_THREAD_LOCAL));
aclnnAdd(wsAddr0, addWsSize0, addExecutor0, subStream);
// 子model end capture
CHECK_ERROR(aclmdlRICaptureEnd(subStream, &subModels[0]));

aclnnMul(mulWorkspaceAddr, mulWorkspaceSize, mulExecutor, stream);
// 父model end capture
CHECK_ERROR(aclmdlRICaptureEnd(stream, &parentModelRI));

// replay
CHECK_ERROR(aclmdlRIExecuteAsync(parentModelRI, execStream));
CHECK_ERROR(aclrtSynchronizeStream(execStream));
```

Kernel代码

```
__global__ void setHandle(conditionalHandle handle)
{
    unsigned int value = someCondition ? 1 : 0;
    SetConditional(handle, value);
}
```


CANN

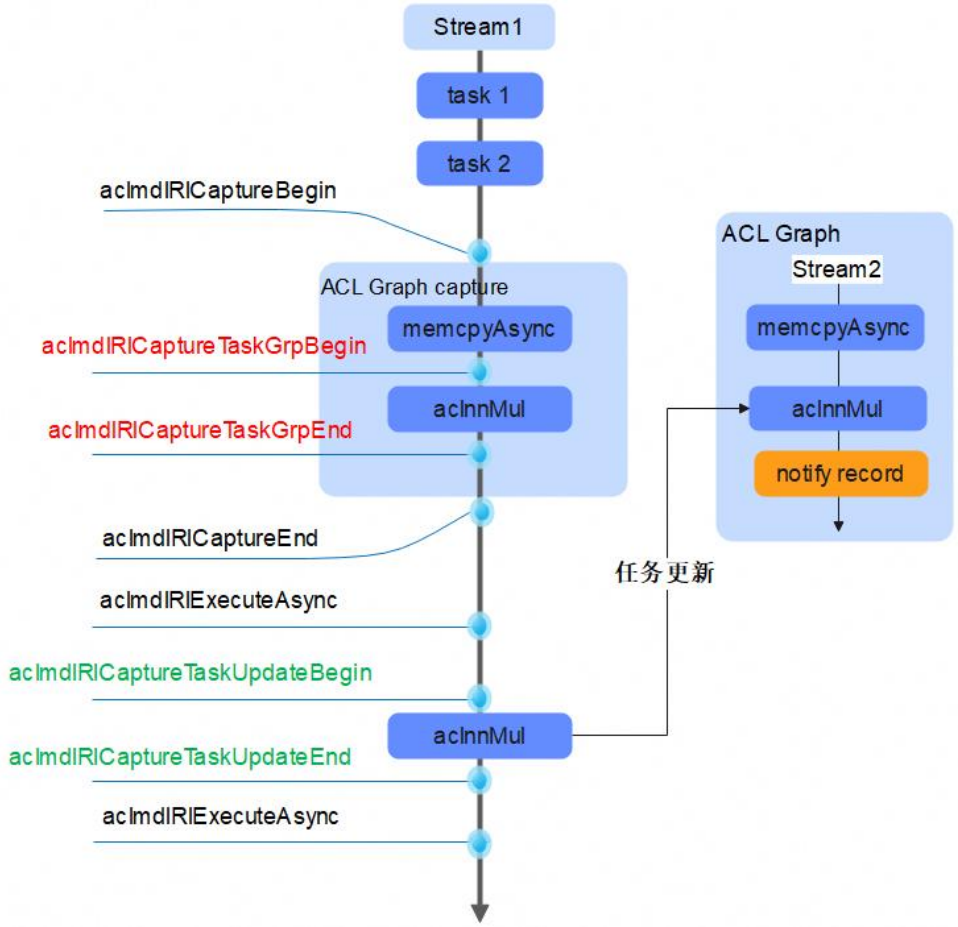
task更新原理

捕获过程中，在某条stream上下发aclmdlRICaptureTaskGrpBegin与aclmdlRICaptureTaskGrpEnd，并把在其之间下发的task组合成一个taskGroup，通过handle的形式返回给用户，用户可以操作这个handle，并下发更新任务，以达到任务更新的目的。

```
// capture阶段
// 开始捕获
CHECK_ERROR(aclmdlRICaptureBegin(stream,
    ACL_MODEL_RI_CAPTURE_MODE_GLOBAL));
CHECK_ERROR(aclrtMemcpyAsync(device, size, host, size,
    ACL_MEMCPY_HOST_TO_DEVICE, stream));

CHECK_ERROR(aclmdlRICaptureTaskGrpBegin(stream));
aclnnAdd(addWorkspaceAddr, addWorkspaceSize, addExecutor, stream);
// 后续可以通过句柄handle进行task更新
CHECK_ERROR(aclmdlRICaptureTaskGrpEnd(stream, &handle));

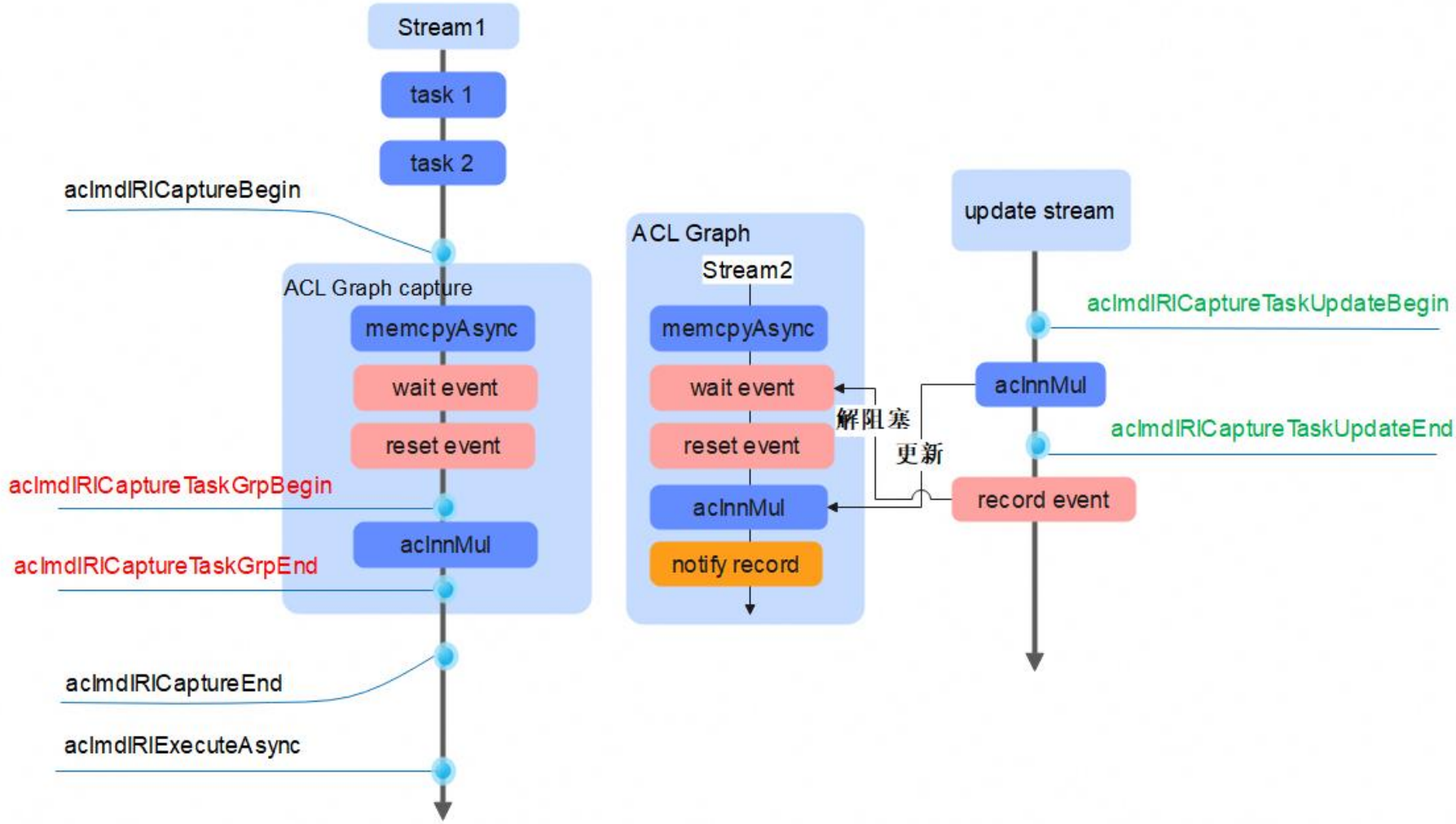
// 捕获结束
CHECK_ERROR(aclmdlRICaptureEnd(stream, &modelRI));
// 通过handle对 handle内task 进行更新
CHECK_ERROR(aclmdlRICaptureTaskUpdateBegin(updateStream, handle));
// 按handle中的task顺序，依次更新
aclnnAdd(updateAddWorkspaceAddr, updateAddWorkspaceSize,
    updateAddExecutor, updateStream);
CHECK_ERROR(aclmdlRICaptureTaskUpdateEnd(updateStream));
```



CANN

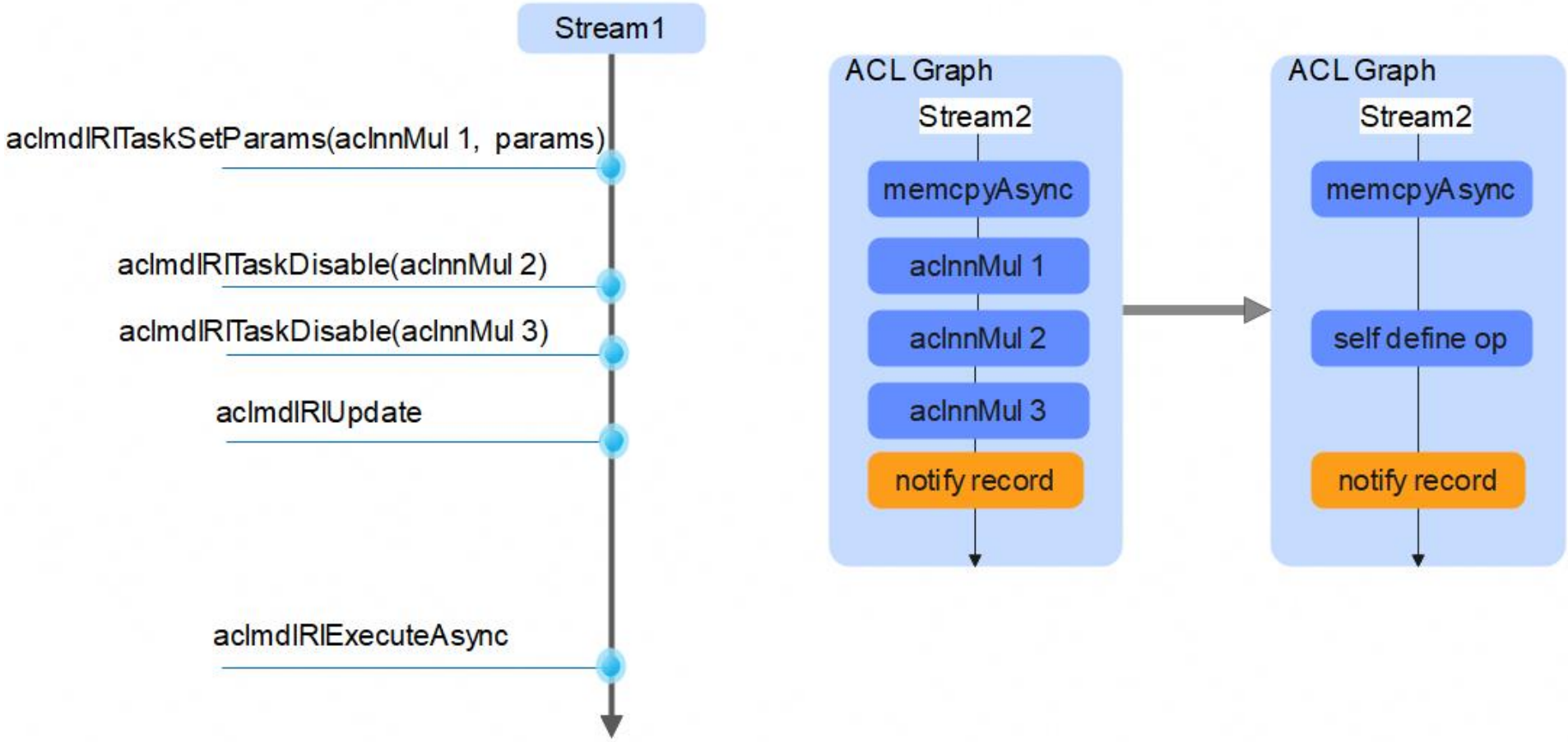
task异步更新

通过在task group前插入External event wait达到模型执行与task更新同步并行执行的目的。
External event wait任务会等模型外的record event任务，由调用者保证record event与更新任务的时序。



task融合更新

由于kernel内的调用路径比NPU更短，性能更好，当存在大量连续AI Core算子时，可以把这些算子融合为一个大算子，在融合算子内部进行逐个调度，减少NPU调度开销，调度收益可达15%以上。



CANN

回顾

模型	Eager模式	aclgraph_ex	加速比
Qwen3-30B	150.33ms	20.2ms	7.4x
DeepSeek-V3.1	185.6ms	41.2ms	4.5x

目录

Part 1 整体方案介绍

Part 2 关键设计介绍

Part 3 未来演进方向

未来展望

未来将围绕易用性与可集成性，完善及丰富ACL Graph的功能，持续进行性能优化。
比如task动态更新、 Graph-Autofusion等功能。
也欢迎大家一起参与cann/runtime的发展，共同打造坚实的昇腾算力底座。

Task静态更新伪代码

```
aclmdlRIGetStreams(modelRI, streams, numStreams); // 查询指定图中有几条流
aclmdlRIGetTasks(modelRI, tasks, numTasks); // 查询流上的任务
aclmdlRITaskGetParams(task, params); // 查询指定task的参数
aclmdlRITaskSetParams(task, params); // 对指定task配置新的参数
aclmdlRITaskDisable(stream, &modelRI); // 从图去使能指定的task
aclmdlRIUpdate(modelRI); // 更新图，再进行模型执行
aclmdlRIExecute(modelRI, timeout); // 模型执行
```

Task动态更新伪代码

```
aclmdlRIGetStreams(modelRI, streams, numStreams); // 查询指定图中有几条流
aclmdlRIGetTasks(modelRI, tasks, numTasks); // 查询流上的任务
aclmdlRITaskGetParams(task, params); // 查询指定task的参数
aclmdlRITaskSetParams(task, params); // 对指定task配置新的参数
aclmdlRITaskDisable(stream, &modelRI); // 从图去使能指定的task
aclmdlRIExecute(modelRI, timeout); // 模型执行

// 模型实际未执行完情况下，重复task更新
aclmdlRITaskSetParams(task, params); // 动态更新场景下，配置的参数仅对下一次模型执行生效
aclmdlRITaskDisable(stream, &modelRI); // 从图去使能指定的task
aclmdlRIExecute(modelRI, timeout); // 模型执行
```

CANN 运行时开源访问地址

运行时开源仓地址：
https://gitcode.com/cann/runtime

API文档：
https://gitcode.com/cann/runtime/tree/master/docs

Sample：
https://gitcode.com/cann/runtime/tree/master/example



Runtime 助手

扫二维码，添加我为朋友。

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann